

Sequence Alignment/Map Format Specification

The SAM/BAM Format Specification Working Group

4 Jun 2026

The master version of this document can be found at <https://github.com/samtools/hts-specs>.

This printing is version 1bd8dca from that repository, last modified on the date shown above.

1 The SAM Format Specification

SAM stands for Sequence Alignment/Map format. It is a TAB-delimited text format consisting of a header section, which is optional, and an alignment section. If present, the header must be prior to the alignments. Header lines start with '@', while alignment lines do not. Each alignment line has 11 mandatory fields for essential alignment information such as mapping position, and variable number of optional fields for flexible or aligner specific information.

Hello, world!

This specification is for version 1.6 of the SAM and BAM formats. Each SAM and BAM file may optionally specify the version being used via the @HD VN tag. For full version history see Appendix B.

SAM file contents are 7-bit US-ASCII, except for certain field values as individually specified which may contain other Unicode characters encoded in UTF-8. Alternatively and equivalently, SAM files are encoded in UTF-8 but non-ASCII characters are permitted only within certain field values as explicitly specified in the descriptions of those fields.¹

Where it makes a difference, SAM file contents should be read and written using the POSIX / C locale. For example, floating-point values in SAM always use '.' for the decimal-point character.

The regular expressions in this specification are written using the POSIX / IEEE Std 1003.1 extended syntax.

1.1 An example

Suppose we have the following alignment with bases in lowercase clipped from the alignment. Read r001/1 and r001/2 constitute a read pair; r003 is a chimeric read; r004 represents a split alignment.

```
Coor      12345678901234 5678901234567890123456789012345
ref      AGCATGTTAGATAA**GATAGCTGTGCTAGTAGGCAGTCAGCGCCAT

+r001/1      TTAGATAAAGGATA*CTG
+r002      aaaAGATAA*GGATA
+r003      gcctaAGCTAA
+r004      ATAGCT.....TCAGC
-r003      ttagctTAGGC
-r001/2      CAGCGGCAT
```

¹Hence in particular SAM files must not begin with a byte order mark (BOM) and lines of text are delimited by ASCII line terminator characters only. In addition to the local platform's text file line termination conventions, implementations may wish to support LF and CRLF for interoperability with other platforms.

The corresponding SAM format is:²

```
@HD VN:1.6 S0:coordinate
@SQ SN:ref LN:45
r001 99 ref 7 30 8M2I4M1D3M = 37 39 TTAGATAAAGGATACTG *
r002 0 ref 9 30 3S6M1P1I4M * 0 0 AAAAGATAAAGGATA *
r003 0 ref 9 30 5S6M * 0 0 GCCTAAGCTAA * SA:Z:ref,29,-,6H5M,17,0;
r004 0 ref 16 30 6M14N5M * 0 0 ATAGCTTCAGC *
r003 2064 ref 29 17 6H5M * 0 0 TAGGC * SA:Z:ref,9,+,5S6M,30,1;
r001 147 ref 37 30 9M = 7 -39 CAGCGGCAT * NM:i:1
```

1.2 Terminologies and Concepts

Template A DNA/RNA sequence part of which is sequenced on a sequencing machine or assembled from raw sequences.

Segment A contiguous sequence or subsequence.

Read A raw sequence that comes off a sequencing machine. A read may consist of multiple segments. For sequencing data, reads are indexed by the order in which they are sequenced.

Linear alignment An alignment of a read to a single reference sequence that may include insertions, deletions, skips and clipping, but may not include direction changes (i.e., one portion of the alignment on forward strand and another portion of alignment on reverse strand). A linear alignment can be represented in a single SAM record.

Chimeric alignment An alignment of a read that cannot be represented as a linear alignment. A chimeric alignment is represented as a set of linear alignments that do not have large overlaps. Typically, one of the linear alignments in a chimeric alignment is considered the “representative” alignment, and the others are called “supplementary” and are distinguished by the supplementary alignment flag. All the SAM records in a chimeric alignment have the same QNAME and the same values for 0x40 and 0x80 flags (see Section 1.4). The decision regarding which linear alignment is representative is arbitrary.

Read alignment A linear alignment or a chimeric alignment that is the complete representation of the alignment of the read.

Multiple mapping The correct placement of a read may be ambiguous, e.g., due to repeats. In this case, there may be multiple read alignments for the same read. One of these alignments is considered primary. All the other alignments have the secondary alignment flag set in the SAM records that represent them. All the SAM records have the same QNAME and the same values for 0x40 and 0x80 flags. Typically the alignment designated primary is the best alignment, but the decision may be arbitrary.³

1-based coordinate system A coordinate system where the first base of a sequence is one. In this coordinate system, a region is specified by a closed interval. For example, the region between the 3rd and the 7th bases inclusive is [3, 7]. The SAM, VCF, GFF and Wiggle formats are using the 1-based coordinate system.

0-based coordinate system A coordinate system where the first base of a sequence is zero. In this coordinate system, a region is specified by a half-closed-half-open interval. For example, the region between the 3rd and the 7th bases inclusive is [2, 7). The BAM, BCFv2, BED, and PSL formats are using the 0-based coordinate system.

²The values in the FLAG column correspond to bitwise flags as follows: 99 = 0x63: first/next is reverse-complemented/ properly aligned/multiple segments; 0: no flags set, thus a mapped single segment; 2064 = 0x810: supplementary/reverse-complemented; 147 = 0x93: last (second of a pair)/reverse-complemented/ properly aligned/multiple segments.

Phred scale Given a probability $0 < p \leq 1$, the phred scale of p equals $-10 \log_{10} p$, rounded to the closest integer.

1.2.1 Character set restrictions

Reference sequence names, CIGAR strings, and several other field types are used as values or parts of values of other fields in SAM and related formats such as VCF. To ensure that these other fields' representations are unambiguous, these field types disallow particular delimiter characters.

Query or read names may contain any printable ASCII characters in the range `[!~]` apart from `@`, so that SAM alignment lines can be easily distinguished from header lines. (They are also limited in length.)

Reference sequence names may contain any printable ASCII characters in the range `[!~]` apart from backslashes, commas, quotation marks, and brackets—i.e., apart from `\ , " ' ( ) [ ] { } < >`—and may not start with `*` or `=`.⁴

Thus they match the following regular expression:

```
[0-9A-Za-z!#$%&+./:;?@^_|\~-] [0-9A-Za-z!#$%&*+./:;=?@^_|\~-]*
```

For clarity, elsewhere in this specification we write this set of allowed characters as a character class `[:rname:]` and extend the POSIX regular expression notation to use `^*=` to indicate the omission of `*` and `=` from the character class. Thus this regular expression can be written more clearly as `[:rname:^*=] [:rname:]*`.

1.3 The header section

Each header line begins with the character `@` followed by one of the two-letter header record type codes defined in this section. In the header, each line is TAB-delimited and, apart from `@CO` lines, each data field follows a format `TAG:VALUE` where TAG is a two-character string that defines the format and content of VALUE. Thus header lines match `/^@(\HD|SQ|RG|PG) (\t[A-Za-z][A-Za-z0-9]: [ -~]+)+$ /` or `/^@CO\t.*$ /`. Within each (non-`@CO`) header line, no field tag may appear more than once and the order in which the fields appear is not significant.

The following table describes the header record types that may be used and their predefined tags. Tags listed with `*` are required; e.g., every `@SQ` header line must have SN and LN fields. As with alignment optional fields (see Section 1.5), you can freely add new tags for further data fields. Tags containing lowercase letters are reserved for local use and will not be formally defined in any future version of this specification.⁵

Tag	Description
<code>@HD</code>	File-level metadata. Optional. If present, there must be only one <code>@HD</code> line and it must be the first line of the file.
<code>VN*</code>	Format version. <i>Accepted format:</i> <code>/^[0-9]+\.[0-9]+\$/</code> .

³Chimeric alignments are primarily caused by structural variations, gene fusions, misassemblies, RNA-seq or experimental protocols. They are more frequent given longer reads. For a chimeric alignment, the linear alignments constituting the alignment are largely non-overlapping; each linear alignment may have high mapping quality and is informative in SNP/INDEL calling. In contrast, multiple mappings are caused primarily by repeats. They are less frequent given longer reads. If a read has multiple mappings, all these mappings are almost entirely overlapping with each other; except the single-best optimal mapping, all the other mappings get mapping quality $< Q3$ and are ignored by most SNP/INDEL callers.

⁴Characters that are *not* disallowed include `|`, which historically appeared in reference names derived from NCBI FASTA files, and `:`, which appears in HLA allele names. Appendix A describes approaches for parsing `name[:begin-end]` region notation unambiguously even though *name* may itself contain colons.

⁵Best practice is to use lowercase tags while designing and experimenting with new data field tags or for fields of local interest only. For new tags that are of general interest, raise an `hts-specs` issue or email `samtools-devel@lists.sourceforge.net` to have an uppercase equivalent added to the specification. This way collisions of the same uppercase tag being used with different meanings can be avoided.

SO	Sorting order of alignments. <i>Valid values:</i> unknown (default), unsorted , queryname and coordinate . For coordinate sort, the major sort key is the RNAME field, with order defined by the order of @SQ lines in the header. The minor sort key is the POS field. For alignments with equal RNAME and POS, order is arbitrary. All alignments with '*' in RNAME field follow alignments with some other value but otherwise are in arbitrary order. For queryname sort, no explicit requirement is made regarding the ordering other than that it be applied consistently throughout the entire file. ⁶
GO	Grouping of alignments, indicating that similar alignment records are grouped together but the file is not necessarily sorted overall. <i>Valid values:</i> none (default), query (alignments are grouped by QNAME), and reference (alignments are grouped by RNAME/POS).
SS	Sub-sorting order of alignments. Valid values are of the form <i>sort-order:sub-sort</i> , where <i>sort-order</i> is the same value stored in the SO tag and <i>sub-sort</i> is an implementation-dependent colon-separated string further describing the sort order, but with some predefined terms defined in Section 1.3.1. For example, if an algorithm relies on a coordinate sort that, at each coordinate, is further sorted by query name then the header could contain @HD SO:coordinate SS:coordinate:queryname. ⁷ If the primary sort is not one of the predefined primary sort orders, then unsorted should be used and the sub-sort is effectively the major sort. For example, if sorted by an auxiliary tag MI then by coordinate then the header could contain @HD SO:unsorted SS:unsorted:MI:coordinate. <i>Regular expression:</i> (coordinate queryname unsorted)(:[A-Za-z0-9_-]+)+
@SQ	Reference sequence dictionary. The order of @SQ lines defines the alignment sorting order.
SN*	Reference sequence name. The SN tags and all individual AN names in all @SQ lines must be distinct. The value of this field is used in the alignment records in RNAME and RNEXT fields. <i>Regular expression:</i> [:rname:^*=] [:rname:]*
LN*	Reference sequence length. <i>Range:</i> [1, 2 ³¹ - 1]
AH	Indicates that this sequence is an alternate locus. ⁸ The value is the locus in the primary assembly for which this sequence is an alternative, in the format ' <i>chr:start-end</i> ', ' <i>chr</i> ' (if known), or '*' (if unknown), where ' <i>chr</i> ' is a sequence in the primary assembly. Must not be present on sequences in the primary assembly.
AN	Alternative reference sequence names. A comma-separated list of alternative names that tools may use when referring to this reference sequence. ⁹ These alternative names are not used elsewhere within the SAM file; in particular, they must not appear in alignment records' RNAME or RNEXT fields. <i>Regular expression:</i> name(,name)* where <i>name</i> is [:rname:^*=] [:rname:]*
AS	Genome assembly identifier.
DS	Description. UTF-8 encoding may be used.
M5	MD5 checksum of the sequence. See Section 1.3.2
SP	Species.
TP	Molecule topology. <i>Valid values:</i> linear (default) and circular . ¹⁰
UR	URI of the sequence. This value may start with one of the standard protocols, e.g., ' http: ' or ' ftp: '. If it does not start with one of these protocols, it is assumed to be a file-system path.
@RG	Read group. Unordered multiple @RG lines are allowed.
ID*	Read group identifier. Each @RG line must have a unique ID. The value of ID is used in the RG tags of alignment records. Must be unique among all read groups in header section. Read group IDs may be modified when merging SAM files in order to handle collisions.
BC	Barcode sequence identifying the sample or library. This value is the expected barcode bases as read by the sequencing machine in the absence of errors. If there are several barcodes for the sample/library (e.g., one on each end of the template), the recommended implementation concatenates all the barcodes separating them with hyphens ('-').
CN	Name of sequencing center producing the read.
DS	Description. UTF-8 encoding may be used.

⁶It is known that widely used software libraries have differing definitions of the queryname sort order, meaning care should be taken when operating on multiple files of varying provenance. Tools may wish to use the sub-sort field to explicitly distinguish between natural and lexicographical ordering. See Section 1.3.1.

⁷The repetition of *sort-order* enables a limited form of validation. For example, @HD SO:queryname SS:coordinate:TLEN would indicate that the data has been re-sorted (by query name) by a non-SS-aware tool and the SS field should be ignored.

⁸See <https://www.ncbi.nlm.nih.gov/grc/help/definitions> for descriptions of *alternate locus* and *primary assembly*.

⁹For example, given '@SQ SN:MT AN:chrMT,M,chrM LN:16569 TP:circular', tools can ensure that a user's request for any of 'MT', 'chrMT', 'M', or 'chrM' succeeds and refers to the same sequence.

¹⁰The previous footnote's example identifies MT as a circular chromosome. The TP field is often omitted, which implies linear.

DT	Date the run was produced (ISO8601 date or date/time).
FO	Flow order. The array of nucleotide bases that correspond to the nucleotides used for each flow of each read. Multi-base flows are encoded in IUPAC format, and non-nucleotide flows by various other characters. <i>Format: /* [ACMGRSVTWYHKDBN] +/</i>
KS	The array of nucleotide bases that correspond to the key sequence of each read.
LB	Library.
PG	Programs used for processing the read group.
PI	Predicted median insert size, rounded to the nearest integer.
PL	Platform/technology used to produce the reads. <i>Valid values: CAPILLARY, DNBSEQ (MGI/BGI), ELEMENT, HELICOS, ILLUMINA, IONTORRENT, LS454, ONT (Oxford Nanopore), PACBIO (Pacific Biosciences), SINGULAR, SOLID, and ULTIMA.</i> ¹¹ This field should be omitted when the technology is not in this list (though the PM field may still be present in this case) or is unknown.
PM	Platform model. Free-form text providing further details of the platform/technology used.
PU	Platform unit (e.g., flowcell-barcode.lane for Illumina or slide for SOLiD). Unique identifier.
SM	Sample. Use pool name where a pool is being sequenced.
@PG	Program.
ID*	Program record identifier. Each @PG line must have a unique ID. The value of ID is used in the alignment PG tag and PP tags of other @PG lines. PG IDs may be modified when merging SAM files in order to handle collisions.
PN	Program name
CL	Command line. UTF-8 encoding may be used.
PP	Previous @PG-ID. Must match another @PG header's ID tag. @PG records may be chained using PP tag, with the last record in the chain having no PP tag. This chain defines the order of programs that have been applied to the alignment. PP values may be modified when merging SAM files in order to handle collisions of PG IDs. The first PG record in a chain (i.e., the one referred to by the PG tag in a SAM record) describes the most recent program that operated on the SAM record. The next PG record in the chain describes the next most recent program that operated on the SAM record. The PG ID on a SAM record is not required to refer to the newest PG record in a chain. It may refer to any PG record in a chain, implying that the SAM record has been operated on by the program in that PG record, and the program(s) referred to via the PP tag.
DS	Description. UTF-8 encoding may be used.
VN	Program version
@CO	One-line text comment. Unordered multiple @CO lines are allowed. UTF-8 encoding may be used.

1.3.1 Defined sub-sort terms

While the **SS** sub-sort field allows implementation-defined keywords, some terms are predefined with specific meanings.

lexicographical sort order is defined as a character-based dictionary sort with the character order as defined by the POSIX C locale. For example “abc”, “abc17”, “abc5”, “abc59” and “abcd” are in lexicographical order.

natural sort order is similar to lexicographical order except that runs of adjacent digits are considered to be numbers embedded within the text string, ordered numerically when compared to each other and ordered as single digits when compared to the surrounding non-digit characters. Runs that differ only in the number of leading zeros (thus are numerically tied) are ordered by more-zeros coming before fewer-zeros. The characters ‘-’ and ‘.’ are considered as ordinary characters, so apparently negative or fractional values are not treated as part of an embedded number. For example, “abc”, “abc+5”, “abc-5”, “abc.d”, “abc03”, “abc5”, “abc008”, “abc08”, “abc8”, “abc17”, “abc17.+”, “abc17.2”, “abc17.d”, “abc59” and “abcd” are in natural order.

¹¹The PL value should be written in uppercase exactly as shown in this list of valid values. Tools should also accept lowercase when reading the @RG PL field, due to the existence of public data files with lowercase PL values.

umi is a lexicographical sort by the UMI tag. The MI tag should be used for comparing UMIs. The RX tag may be used in its absence but is not guaranteed to be unique across multiple libraries.

1.3.2 Reference MD5 calculation

The M5 tag on @SQ lines allows reference sequences to be uniquely identified through the MD5 digest of the sequence itself. As the digest is based on the sequence and nothing else, it can help resolve ambiguities with reference naming. For example, it allows a quick way of checking that references named ‘1’, ‘Chr1’ and ‘chr1’ in different files are in fact the same.

The reference sequence must be in the 7-bit US-ASCII character set. All valid reference bases can be represented in this set, and it avoids the problem of determining exactly which 8-bit representation may have been used. Padding characters (See Section 3.2) must be represented only using the ‘*’ character.

The digest is calculated as follows:

- All characters outside of the inclusive range 33 (!) to 126 (~) are stripped out. This removes all unprintable and whitespace characters including spaces and new lines. Everything else is retained, even if not a legal nucleotide code.
- All lowercase characters are converted to uppercase. This operation is equivalent to calling *toupper()* on characters in the *POSIX locale*.
- The MD5 digest is calculated as described in *RFC 1321* and presented as a 32 character lowercase hexadecimal number.

As an example, if the reference contains the following characters (including spaces):

```
ACGT ACGT ACGT
acgt acgt acgt
... 12345 !!!
```

then the digest is that of the string ACGTACGTACGTACGTACGTACGT...12345!!! and the resulting tag would be M5:dfabdbb36e239a6da88957841f32b8e4.

In padded SAM files, the padding bases should be inserted into the reference as ‘*’ characters. Taking the example in Section 3.2, the padded version of the reference is

```
AGCATGTTAGATAA**GATAGCTGTGCTAGTAGGCAGTCAGCGCCAT
```

and the corresponding tag is M5:caad65b937c4bc0b33c08f62a9fb5411.

1.4 The alignment section: mandatory fields

In the SAM format, each alignment line typically represents the linear alignment of a segment. Each line consists of 11 or more TAB-separated fields. The first eleven fields are always present and in the order shown below; if the information represented by any of these fields is unavailable, that field’s value will be a placeholder, either ‘0’ or ‘*’ as determined by the field’s type. The following table gives an overview of these mandatory fields in the SAM format:

Col	Field	Type	Regexp/Range	Brief description
1	QNAME	String	[!-?A-~]{1,254}	Query template NAME
2	FLAG	Int	[0, 2 ¹⁶ – 1]	bitwise FLAG
3	RNAME	String	* [:rname:^*=] [:rname:]*	Reference sequence NAME ¹²
4	POS	Int	[0, 2 ³¹ – 1]	1-based leftmost mapping POSition
5	MAPQ	Int	[0, 2 ⁸ – 1]	MAPping Quality
6	CIGAR	String	* ([0-9]+[MIDNSHP=X])+	CIGAR string
7	RNEXT	String	* = [:rname:^*=] [:rname:]*	Reference name of the mate/next read
8	PNEXT	Int	[0, 2 ³¹ – 1]	Position of the mate/next read
9	TLEN	Int	[–2 ³¹ + 1, 2 ³¹ – 1]	observed Template LENGth
10	SEQ	String	* [A-Za-z=]+	segment SEQUENCE
11	QUAL	String	[!-~]+	ASCII of Phred-scaled base QUALity+33

All mapped segments in alignment lines are represented on the forward genomic strand. For segments that have been mapped to the reverse strand, the recorded **SEQ** is reverse complemented from the original unmapped sequence and **CIGAR**, **QUAL**, and strand-sensitive optional fields are reversed and thus recorded consistently with the sequence bases as represented.

1. **QNAME**: Query template NAME. Reads/segments having identical **QNAME** are regarded to come from the same template. A **QNAME** ‘*’ indicates the information is unavailable. In a SAM file, a read may occupy multiple alignment lines, when its alignment is chimeric or when multiple mappings are given.
2. **FLAG**: Combination of bitwise **FLAGS**.¹³ Each bit is explained in the following table:

Bit	Description
1	0x1 template having multiple segments in sequencing
2	0x2 each segment properly aligned according to the aligner
4	0x4 segment unmapped
8	0x8 next segment in the template unmapped
16	0x10 SEQ being reverse complemented
32	0x20 SEQ of the next segment in the template being reverse complemented
64	0x40 the first segment in the template
128	0x80 the last segment in the template
256	0x100 secondary alignment
512	0x200 not passing filters, such as platform/vendor quality controls
1024	0x400 PCR or optical duplicate
2048	0x800 supplementary alignment

- For each read/contig in a SAM file, it is required that one and only one line associated with the read satisfies ‘**FLAG & 0x900 == 0**’. This line is called the *primary line* of the read.
- Bit 0x100 marks the alignment not to be used in certain analyses when the tools in use are aware of this bit. It is typically used to flag alternative mappings when multiple mappings are presented in a SAM.
- Bit 0x800 indicates that the corresponding alignment line is part of a chimeric alignment. A line flagged with 0x800 is called as a *supplementary line*.
- Bit 0x4 is the only reliable place to tell whether the read is unmapped. If 0x4 is set, no assumptions can be made about **RNAME**, **POS**, **CIGAR**, **MAPQ**, and bits 0x2, 0x100, and 0x800.
- Bit 0x10 indicates whether **SEQ** has been reverse complemented and **QUAL** reversed. When bit 0x4 is unset, this corresponds to the strand to which the segment has been mapped: bit 0x10 unset indicates the forward strand, while set indicates the reverse strand. When 0x4 is set, this indicates whether the unmapped read is stored in its original orientation as it came off the sequencing machine.
- Bits 0x40 and 0x80 reflect the read ordering within each template inherent in the sequencing technology used.¹⁴ If 0x40 and 0x80 are both set, the read is part of a linear template, but it is neither the first nor the last read. If both 0x40 and 0x80 are unset, the index of the read in the template is unknown. This may happen for a non-linear template or when this information is lost during data processing.
- If 0x1 is unset, no assumptions can be made about 0x2, 0x8, 0x20, 0x40 and 0x80.
- Bits that are not listed in the table are reserved for future use. They should not be set when writing and should be ignored on reading by current software.

¹²Reference sequence names may contain any printable ASCII characters with the exception of certain punctuation characters, and may not start with ‘*’ or ‘=’. See Section 1.2.1 for details and an explanation of the `[:rname:]` notation.

¹³The manipulation of bitwise flags is described at Wikipedia (see “Bit field”) and elsewhere.

¹⁴For example, in Illumina paired-end sequencing, **first** (0x40) corresponds to the R1 ‘forward’ read and **last** (0x80) to the R2 ‘reverse’ read. (Despite the terminology, this is unrelated to the segments’ orientations when they are mapped: either, neither, or both may have their **reverse** flag bits (0x10) set after mapping.)

3. **RNAME**: Reference sequence NAME of the alignment. If **@SQ** header lines are present, **RNAME** (if not ‘*’) must be present in one of the **SQ-SN** tag. An unmapped segment without coordinate has a ‘*’ at this field. However, an unmapped segment may also have an ordinary coordinate such that it can be placed at a desired position after sorting. If **RNAME** is ‘*’, no assumptions can be made about **POS** and **CIGAR**.
4. **POS**: 1-based leftmost mapping POSition of the first **CIGAR** operation that “consumes” a reference base (see table below). The first base in a reference sequence has coordinate 1. **POS** is set as 0 for an unmapped read without coordinate. If **POS** is 0, no assumptions can be made about **RNAME** and **CIGAR**.
5. **MAPQ**: MAPping Quality. It equals $-10 \log_{10} \Pr\{\text{mapping position is wrong}\}$, rounded to the nearest integer. A value 255 indicates that the mapping quality is not available.
6. **CIGAR**: **CIGAR** string. The **CIGAR** operations are given in the following table (set ‘*’ if unavailable):

Op	BAM	Description	Consumes query	Consumes reference
M	0	alignment match (can be a sequence match or mismatch)	yes	yes
I	1	insertion to the reference	yes	no
D	2	deletion from the reference	no	yes
N	3	skipped region from the reference	no	yes
S	4	soft clipping (clipped sequences present in SEQ)	yes	no
H	5	hard clipping (clipped sequences NOT present in SEQ)	no	no
P	6	padding (silent deletion from padded reference)	no	no
=	7	sequence match	yes	yes
X	8	sequence mismatch	yes	yes

- “Consumes query” and “consumes reference” indicate whether the **CIGAR** operation causes the alignment to step along the query sequence and the reference sequence respectively.
 - **H** can only be present as the first and/or last operation.
 - **S** may only have **H** operations between them and the ends of the **CIGAR** string.
 - For mRNA-to-genome alignment, an **N** operation represents an intron. For other types of alignments, the interpretation of **N** is not defined.
 - Sum of lengths of the **M/I/S/=/X** operations shall equal the length of **SEQ**.
7. **RNEXT**: Reference sequence name of the primary alignment of the **NEXT** read in the template. For the last read, the next read is the first read in the template. If **@SQ** header lines are present, **RNEXT** (if not ‘*’ or ‘=’) must be present in one of the **SQ-SN** tag. This field is set as ‘*’ when the information is unavailable, and set as ‘=’ if **RNEXT** is identical **RNAME**. If not ‘=’ and the next read in the template has one primary mapping (see also bit 0x100 in **FLAG**), this field is identical to **RNAME** at the primary line of the next read. If **RNEXT** is ‘*’, no assumptions can be made on **PNEXT** and bit 0x20.
 8. **PNEXT**: 1-based Position of the primary alignment of the **NEXT** read in the template. Set as 0 when the information is unavailable. This field equals **POS** at the primary line of the next read. If **PNEXT** is 0, no assumptions can be made on **RNEXT** and bit 0x20.
 9. **TLEN**: signed observed Template LENgth. For primary reads where the primary alignments of all reads in the template are mapped to the same reference sequence, the absolute value of **TLEN** equals the distance between the mapped end of the template and the mapped start of the template, inclusively (i.e., $\text{end} - \text{start} + 1$).¹⁵ Note that *mapped base* is defined to be one that aligns to the reference as described by **CIGAR**, hence excludes soft-clipped bases. The **TLEN** field is positive for the leftmost segment of the template, negative for the rightmost, and the sign for any middle segment is undefined. If segments cover the same coordinates then the choice of which is leftmost and rightmost is arbitrary,

¹⁵Thus a segment aligning in the forward direction at base 100 for length 50 and a segment aligning in the reverse direction at base 200 for length 50 indicate the template covers bases 100 to 249 and has length 150.

but the two ends must still have differing signs. It is set as 0 for a single-segment template or when the information is unavailable (e.g., when the first or last segment of a multi-segment template is unmapped or when the two are mapped to different reference sequences).

The intention of this field is to indicate where the other end of the template has been aligned without needing to read the remainder of the SAM file. Unfortunately there has been no clear consensus on the definitions of the template mapped start and end. Thus the exact definitions are implementation-defined.¹⁶

10. **SEQ**: segment SEQUENCE. This field can be a '*' when the sequence is not stored. If not a '*', the length of the sequence must equal the sum of lengths of M/I/S/=/X operations in CIGAR. An '=' denotes the base is identical to the reference base. No assumptions can be made on the letter cases.
11. **QUAL**: ASCII of base QUALity plus 33 (same as the quality string in the Sanger FASTQ format). A base quality is the phred-scaled base error probability which equals $-10 \log_{10} \text{Pr}\{\text{base is wrong}\}$. This field can be a '*' when quality is not stored.¹⁷ If not a '*', SEQ must not be a '*' and the length of the quality string ought to equal the length of SEQ.

1.5 The alignment section: optional fields

All optional fields follow the TAG:TYPE:VALUE format where TAG is a two-character string that matches `/[A-Za-z][A-Za-z0-9]/`. Within each alignment line, no TAG may appear more than once and the order in which the optional fields appear is not significant. A TAG containing lowercase letters is reserved for end users. In an optional field, TYPE is a single case-sensitive letter which defines the format of VALUE:

Type	Regex matching VALUE	Description
A	[!~]	Printable character
i	[+]?[0-9]+	Signed integer ¹⁸
f	[+]?[0-9]*\.[0-9]+([eE][+]?[0-9]+)?	Single-precision floating number
Z	[!~]*	Printable string, including space
H	([0-9A-F][0-9A-F])*	Byte array in the Hex format ¹⁹
B	[cCsSiIf](,[+]?[0-9]*\.[0-9]+([eE][+]?[0-9]+)?)*	Integer or numeric array

For an integer or numeric array (type 'B'), the first letter indicates the type of numbers in the following comma separated array. The letter can be one of 'cCsSiIf', corresponding to `int8_t` (signed 8-bit integer), `uint8_t` (unsigned 8-bit integer), `int16_t`, `uint16_t`, `int32_t`, `uint32_t` and `float`, respectively.²⁰ During import/export, the element type may be changed if the new type is also compatible with the array.

Predefined tags are described in the separate *Sequence Alignment/Map Optional Fields Specification*.²¹ See that document for details of existing standard tag fields and conventions around creating new tags that may be of general interest. Tags starting with 'X', 'Y' or 'Z' and tags containing lowercase letters in either position are reserved for local use and will not be formally defined in any future version of these specifications.

¹⁶The earliest versions of this specification used 5' to 5' (in original orientation, TLEN#1; dashed parts of the reads indicate soft-clipped bases) while later ones used leftmost to rightmost mapped base (TLEN#2). Note: these two definitions agree in most alignments, but differ in the case of overlaps where the first segment aligns beyond the start of the last segment.



¹⁷In the unlikely case in which SEQ is of length 1 and QUAL is '*', QUAL should be interpreted as quality not stored. Tools should avoid writing a solitary base quality 9 (ASCII '*') to SAM, BAM, or CRAM files for alignment lines where SEQ is of length 1, instead adjusting the base quality to 10 (ASCII '+') to avoid this potential ambiguity.

¹⁸The number of digits in an integer optional field is not explicitly limited in SAM. However, BAM can represent values in the range $[-2^{31}, 2^{32})$, so in practice this is the realistic range of values for SAM's 'i' as well.

¹⁹For example, the six-character Hex string '1AE301' represents the byte array `[0x1a, 0xe3, 0x01]`.

²⁰Explicit typing eases format parsing and helps to reduce the file size when SAM is converted to BAM.

²¹See `SAMtags.pdf` at <https://github.com/samtools/hts-specs>.

2 Recommended Practice for the SAM Format

This section describes the best practice for representing data in the SAM format. They are not required in general, but may be required by a specific software package for it to function properly.

1. The header section
 - 1 The `@HD` line should be present, with either the `SO` tag or the `GO` tag (but not both) specified.
 - 2 The `@SQ` lines should be present if reads have been mapped.
 - 3 When a `RG` tag appears anywhere in the alignment section, there should be a single corresponding `@RG` line with matching `ID` tag in the header.
 - 4 When a `PG` tag appears anywhere in the alignment section, there should be a single corresponding `@PG` line with matching `ID` tag in the header.
2. Adjacent CIGAR operations should be different.
3. No alignments should be assigned mapping quality 255.
4. Unmapped reads
 - 1 For a unmapped paired-end or mate-pair read whose mate is mapped, the unmapped read should have `RNAME` and `POS` identical to its mate.
 - 2 If all segments in a template are unmapped, their `RNAME` should be set as `*` and `POS` as 0.
 - 3 If `POS` plus the sum of lengths of `M/=X/D/N` operations in `CIGAR` exceeds the length specified in the `LN` field of the `@SQ` header line (if exists) with an `SN` equal to `RNAME`, the alignment should be unmapped, unless the reference sequence is circular (see below).
 - 4 Unmapped reads should be stored in the orientation in which they came off the sequencing machine and have their `reverse` flag bit (0x10) correspondingly unset.
5. Multiple mapping
 - 1 When one segment is present in multiple lines to represent a multiple mapping of the segment, only one of these records should have the secondary alignment flag bit (0x100) unset. `RNEXT` and `PNEXT` point to the primary line of the next read in the template.
 - 2 `SEQ` and `QUAL` of secondary alignments should be set to `*` to reduce the file size.
6. Optional tags:
 - 1 If the template has more than 2 segments, the `TC` tag should be present.
 - 2 The `NM` tag should be present.
7. Circular reference sequences

Mappings that cross the coordinate ‘join’ in circular reference sequences (i.e., those whose `@SQ` headers specify `TP:circular`) may be represented as follows:

 - 1 (Preferred) As usual `POS` should be between 1 and the `@SQ` header’s `LN` value, but `POS` plus the sum of the lengths of `M/=X/D/N` `CIGAR` operations may exceed `LN`. Coordinates greater than `LN` are interpreted by subtracting `LN` so that bases at `LN+1`, `LN+2`, `LN+3`, ... are considered to be mapped at positions 1, 2, 3, ...; thus each (1-based) position p is interpreted as $((p - 1) \bmod LN) + 1$.²²
 - 2 Alternatively, such alignments may be split across several records: one record representing the initial portion of the segment ending at `LN`, one representing the final portion starting from 1, and any other records representing additional portions in between spanning the entire reference sequence. One record (chosen arbitrarily) is considered primary and the remainder have their `supplementary` flag bit (0x800) set.

²²The impact of this representation on indexing and random access is yet to be explored by implementations.

8. Annotation dummy reads: These have SEQ set to *, FLAG bits 0x100 and 0x200 set (secondary and filtered), and a CT tag.

- 1 If you wish to store free text in a CT tag, use the *key* value **Note** (uppercase N) to match GFF3.
- 2 Multi-segment annotation (e.g., a gene with introns) should be described with multiple lines in SAM (like a multi-segment read). Where there is a clear biological direction (e.g., a gene), the first segment (FLAG bit 0x40) is used for the first section (e.g., the 5' end of the gene). Thus a GenBank entry location like `complement(join(85052..85354, 85441..85621, 86097..86284))` would have three lines in SAM with a common QNAME:

	FLAG	POS	CIGAR	Optional fields	
The 5' fragment	883 (0x373)	86097	188M	FI:i:1	TC:i:3
Middle fragment	819 (0x333)	85441	181M	FI:i:2	TC:i:3
The 3' fragment	947 (0x3B3)	85052	303M	FI:i:3	TC:i:3

- 3 If converting GFF3 to SAM, store any *key, values* from column 9 in the CT tag, except for the unique ID which is used for the QNAME. GFF3 columns 1 (seqid), 4 (start) and 5 (end) are encoded using SAM columns RNAME, POS and CIGAR to hold the length. GFF3 columns 3 (type) and 7 (strand) are stored explicitly in the CT tag. Remaining GFF3 columns 2 (source), 6 (score), and 8 (phase) are stored in the CT tag using *key* values **FSource**, **FScore** and **FPhase** (uppercase keys are restricted in GFF3, so these names avoid clashes). Split location features are described with multiple lines in GFF3, and similarly become multi-segment dummy reads in SAM, with the RNEXT and PNEXT columns filled in appropriately. In the absence of a convention in SAM/BAM for reads wrapping the origin of a circular genome, any GFF3 feature line wrapping the origin must be split into two segments in SAM.

3 Guide for Describing Assembly Sequences in SAM

3.1 Unpadded versus padded representation

To describe alignments, we can regard the reference sequence with no respect to other alignments against it. Such a reference sequence is called an *unpadded reference*. A position on an unpadded reference, referred to as an *unpadded position*, is not affected by any alignments. When we use unpadded references and positions to describe alignments, we say we are using the *unpadded representation*.

Alternatively, to describe the same alignments, we can modify the reference sequence to contain pads that make room for sequences inserted relative to the reference. A pad is effectively a gap and conventionally represented by an asterisk '*'. A reference sequence containing pads is called a *padded reference*. A position which counts the '*'s is referred to as a *padded position*. A padded reference sequence may be affected by the query alignments and because of gap insertions is typically longer than the unpadded reference. The padded position of one query alignment may be affected by other query alignments.

Unpadded and padded are different representations of the same alignments. They are convertible to each other with no loss of any information. The unpadded representation is more common due to the convenience of a fixed coordinate system, while the padded representation has the advantage that alignments can be simply described by the start and end coordinates without using complex CIGAR strings. SAM traditionally uses the padded representation for *de novo* assembly. The ACE assembly format uses the padded representation exclusively.

3.2 Padded SAM

The SAM format is typically used to describe alignments against an unpadded reference sequence, but it is also able to describe alignments against a padded reference. In the latter case, we say we are using a *padded SAM*. A padded SAM is a valid SAM, but with the difference that the reference and positions in use are padded. There may be more than one way to describe the padded representation. We recommend the following; see also the discussion in Cock *et al.*²³

²³Peter J. A. Cock, James K. Bonfield, Bastien Chevreux, and Heng Li, **SAM/BAM format v1.5 extensions for *de novo* assemblies**, *bioRxiv* 020024; doi:10.1101/020024.

In a padded SAM, alignments and coordinates are described with respect to the padded reference sequence. Unlike traditional padded representations like the ACE file format where pads/gaps are recorded in reads using *'s, we do not write *'s in the SEQ field of the SAM format.²⁴ Instead, we describe pads in the query sequences as deletions from the padded reference using the CIGAR 'D' operation. In a padded SAM, the insertion and padding CIGAR operations ('I' and 'P') are not used because the padded reference already considers all the insertions.

The following shows the padded SAM for the example alignment in Section 1.1. Notably, the length of **ref** is 47 instead of 45. **POS** of the last three alignments are all shifted by 2. **CIGAR** of alignments bridging the 2bp insertion are also changed.

```
@HD VN:1.6 S0:coordinate
@SQ SN:ref LN:47
ref 516 ref 1 0 14M2D31M * 0 0 AGCATGTTAGATAAGATAGCTGTGCTAGTAGGCAGTCAGCGCCAT *
r001 99 ref 7 30 14M1D3M = 39 41 TTAGATAAAGGATACTG *
* 768 ref 8 30 1M * 0 0 * * CT:Z:.;Warning;Note=Ref wrong?
r002 0 ref 9 30 3S6M1D5M * 0 0 AAAAGATAAGGATA * PT:Z:1;4;+;homopolymer
r003 0 ref 9 30 5H6M * 0 0 AGCTAA * NM:i:1
r004 0 ref 18 30 6M14N5M * 0 0 ATAGCTTCAGC *
r003 2064 ref 31 30 6H5M * 0 0 TAGGC * NM:i:0
r001 147 ref 39 30 9M = 7 -41 CAGCGGCAT * NM:i:1
```

Here we also exemplify the recommended practice for storing the reference sequence and the reference annotations in SAM when necessary. For a reference sequence in SAM, **QNAME** should be identical to **RNAME**, **POS** set to 1 and **FLAG** to 516 (filtered and unmapped); for an annotation, **FLAG** should be set to 768 (filtered and secondary) with no restriction to **QNAME**. Dummy reads for annotation would typically have a **CT** tag to hold the annotation information; see the discussion of dummy reads in Section 2. See also the separate *Optional Fields Specification* for full details of the **CT** and **PT** annotation tags.²⁵

²⁴Writing pads/gaps as *'s in the **SEQ** field might have been more convenient, but this caused concerns for backward compatibility.

²⁵See *Annotation and Padding in SAMtags.pdf*.

4 The BAM Format Specification

4.1 The BGZF compression format

BGZF is block compression implemented on top of the standard gzip file format.²⁶ The goal of BGZF is to provide good compression while allowing efficient random access to the BAM file for indexed queries. The BGZF format is ‘gzip compatible’, in the sense that a compliant gzip utility can decompress a BGZF compressed file.²⁷

A BGZF file is a series of concatenated BGZF blocks, each no larger than 64 KiB both before and after compression. Each BGZF block is itself a spec-compliant gzip archive which contains an “extra field” in the format described in RFC1952. The gzip file format allows the inclusion of application-specific extra fields and these are ignored by compliant decompression implementation. The gzip specification also allows gzip files to be concatenated. The result of decompressing concatenated gzip files is the concatenation of the uncompressed data.

Each BGZF block contains a standard gzip file header with the following standard-compliant extensions:

1. The F.EXTRA bit in the header is set to indicate that extra fields are present.
2. The extra field used by BGZF uses the two subfield ID values 66 and 67 (ASCII ‘BC’).
3. The length of the BGZF extra field payload (field LEN in the gzip specification) is 2 (two bytes of payload).
4. The payload of the BGZF extra field is a 16-bit unsigned integer in little endian format. This integer gives the size of the containing BGZF block minus one.

On disk, a complete BGZF file is a series of blocks as shown in the following table. (All integers are little endian as is required by RFC1952.)

Field	Description	Type	Value
<i>List of compression blocks (until the end of the file)</i>			
ID1	gzip IDentifier1	uint8_t	31
ID2	gzip IDentifier2	uint8_t	139
CM	gzip Compression Method	uint8_t	8
FLG	gzip FLaGs	uint8_t	4
MTIME	gzip Modification TIME	uint32_t	
XFL	gzip eXtra FLags	uint8_t	
OS	gzip Operating System	uint8_t	
XLEN	gzip eXtra LENgth	uint16_t	
<i>Extra subfield(s) (total size=XLEN)</i>			
<i>Additional RFC1952 extra subfields if present</i>			
SI1	Subfield Identifier1	uint8_t	66
SI2	Subfield Identifier2	uint8_t	67
SLEN	Subfield LENgth	uint16_t	2
BSIZE	total Block SIZE minus 1	uint16_t	
<i>Additional RFC1952 extra subfields if present</i>			
CDATA	Compressed DATA by zlib::deflate()	uint8_t[BSIZE-XLEN-19]	
CRC32	CRC-32	uint32_t	
ISIZE	Input SIZE (length of uncompressed data)	uint32_t	

The random access method to be described next limits the uncompressed contents of each BGZF block to a maximum of 2^{16} bytes of data. Thus while ISIZE is stored as a uint32_t as per the gzip format, in BGZF it is limited to the range [0, 65536]. BSIZE can represent BGZF block sizes in the range [1, 65536], though typically BSIZE will be rather less than ISIZE due to compression.

²⁶L. Peter Deutsch, **GZIP file format specification version 4.3**, RFC 1952.

²⁷It is worth noting that there is a known bug in the Java GZIPInputStream class that concatenated gzip archives cannot be successfully decompressed by this class. BGZF files can be created and manipulated using the built-in Java util.zip package, but naive use of GZIPInputStream on a BGZF file will not work due to this bug.

4.1.1 Random access

BGZF files support random access through the BAM file index. To achieve this, the BAM file index uses *virtual file offsets* into the BGZF file. Each virtual file offset is an unsigned 64-bit integer, defined as: `coffset<<16|uoffset`, where `coffset` is an unsigned byte offset into the BGZF file to the beginning of a BGZF block, and `uoffset` is an unsigned byte offset into the uncompressed data stream represented by that BGZF block. Virtual file offsets can be compared, but subtraction between virtual file offsets and addition between a virtual offset and an integer are both disallowed.

4.1.2 End-of-file marker

An end-of-file (EOF) trailer or marker block should be written at the end of BGZF files, so that unintended file truncation can be easily detected. The EOF marker block is a particular empty²⁸ BGZF block encoded with the default zlib compression level settings, and consists of the following 28 hexadecimal bytes:

1f 8b 08 04 00 00 00 00 00 00 ff 06 00 42 43 02 00 1b 00 03 00 00 00 00 00 00 00 00 00

The presence of this EOF marker at the end of a BGZF file indicates that the immediately following physical EOF is the end of the file as intended by the program that wrote it. Empty BGZF blocks are not otherwise special; in particular, the presence of an EOF marker block does not by itself signal end of file.²⁹

The absence of this final EOF marker should trigger a warning or error soon after opening a BGZF file where random access is available.³⁰ When reading a BGZF file in sequential streaming fashion, ideally this EOF check should be performed when the end of the stream is reached. Checking that the final BGZF block in the file decompresses to empty or checking that the last 28 bytes of the file are exactly the bytes above are both sufficient tests; each is likely more convenient in different circumstances.

²⁸Empty in the sense of having been formed by compressing a data block of length zero.

²⁹An implementation that supports reopening a BAM file in append mode could produce a file by writing headers and alignment records to it, closing it (adding an EOF marker); then reopening it for append, writing more alignment records, and closing it (adding an EOF marker). The resulting BAM file would contain an embedded insignificant EOF marker block that should be effectively ignored when it is read.

³⁰It is useful to produce a diagnostic at the beginning of reading a file, so that interactive users can abort lengthy analysis of potentially-corrupted files. Of course, this is only possible if the stream in question supports random access.

4.2 The BAM format

BAM is compressed in the BGZF format. All multi-byte numbers in BAM are little-endian, regardless of the machine endianness. The format is formally described in the following table where values in brackets are the default when the corresponding information is not available; an underlined word in uppercase denotes a field in the SAM format.

Field	Description	Type	Value
magic	BAM magic string	char[4]	BAM\1
l_text	Length of the header text, including any NUL padding	uint32_t	$< 2^{31}$
text	Plain header text in SAM; not necessarily NUL-terminated	char[l_text]	
n_ref	# reference sequences	uint32_t	$< 2^{31}$
<i>List of reference information (n=n_ref)</i>			
l_name	Length of the reference name plus 1 (including NUL)	uint32_t	<i>limited</i>
name	Reference sequence name; NUL-terminated	char[l_name]	
l_ref	Length of the reference sequence	uint32_t	$< 2^{31}$
<i>List of alignments (until the end of the file)</i>			
block_size	Total length of the alignment record, excluding this field	uint32_t	<i>limited</i>
refID	Reference sequence ID, $-1 \leq \text{refID} < \text{n_ref}$; -1 for a read without a mapping position	int32_t	[-1]
pos	0-based leftmost coordinate ($= \text{POS} - 1$)	int32_t	[-1]
l_read_name	Length of read_name below ($= \text{length}(\text{QNAME}) + 1$)	uint8_t	
mapq	Mapping quality ($= \text{MAPQ}$)	uint8_t	
bin	BAI index bin, see Section 4.2.1	uint16_t	
n_cigar_op	Number of operations in CIGAR, see Section 4.2.2	uint16_t	
flag	Bitwise flags ($= \text{FLAG}$) ³¹	uint16_t	
l_seq	Length of SEQ	uint32_t	<i>limited</i>
next_refID	Ref-ID of the next segment ($-1 \leq \text{next_refID} < \text{n_ref}$)	int32_t	[-1]
next_pos	0-based leftmost pos of the next segment ($= \text{PNEXT} - 1$)	int32_t	[-1]
tlen	Template length ($= \text{TLEN}$)	int32_t	[0]
read_name	Read name, NUL-terminated (QNAME with trailing '\0') ³²	char[l_read_name]	
cigar	CIGAR: op_len<<4 op. 'MIDNSHP=X'→'012345678'	uint32_t[n_cigar_op]	
seq	4-bit encoded read: 'ACMGRSVTWYHKDBN'→[0,15]. See Section 4.2.3	uint8_t[(l_seq+1)/2]	
qual	Phred-scaled base qualities. See Section 4.2.3	char[l_seq]	
<i>List of auxiliary data (until the end of the alignment block)</i>			
tag	Two-character tag	char[2]	
val_type	Value type: AcCsSiIfZHB, see Section 4.2.4	char	
value	Tag value	(by val_type)	

Most length and count fields described as `uint32_t` have additional constraints on their range: `l_text` $< 2^{31}$ due to implementation limits; `n_ref` $< 2^{31}$ because `refID` and `next_refID` are signed; `l_ref` $< 2^{31}$ because `tlen` is signed; those marked “*limited*” are limited by available memory and the practical size of the data represented well before they are limited by, e.g., Java’s signed 32-bit integer maximum array size.

4.2.1 BIN field calculation

BIN is calculated using the `reg2bin()` function in Section 5.3. For mapped reads this uses `POS-1` (i.e., 0-based left position) and the alignment end point using the alignment length from the CIGAR string. For unmapped reads (e.g., paired-end reads where only one part is mapped, see Section 2) and reads whose CIGAR strings consume no reference bases at all, the alignment is treated as being of length one. Note unmapped reads with `POS 0` (which becomes -1 in BAM) therefore use `reg2bin(-1, 0)` which is computed as 4680.

³¹As noted in Section 1.4, reserved `FLAG` bits should be written as zero and ignored on reading by current software.

³²For backward compatibility, an absent `QNAME` (represented as ‘*’ in SAM) is stored as a C string “*\0”.

4.2.2 N_CIGAR_OP field

With 16 bits, `n_cigar_op` can keep at most 65535 CIGAR operations in BAM files. For an alignment with more CIGAR operations, BAM stores the real CIGAR, encoded the same way as the `cigar` field in BAM, in the `CG` optional tag of type 'B,I', and sets CIGAR to '*kSmN*' as a placeholder, where '*k*' equals `l_seq`, '*m*' is the reference sequence length in the alignment, and 'S' and 'N' are the soft-clipping and reference-clip CIGAR operators, respectively—i.e., in the binary form, `n_cigar_op=2` and `cigar=[k<<4|4,m<<4|3]`. If tag `CG` is present and the first CIGAR operation clips the entire read, a BAM parsing library is expected to update `n_cigar_op` and `cigar` with the real CIGAR stored in the `CG` tag and remove the now-redundant `CG` tag.

4.2.3 SEQ and QUAL encoding

Sequence is encoded in 4-bit values, with adjacent bases packed into the same byte starting with the highest 4 bits first. When `l_seq` is odd the bottom 4 bits of the last byte are undefined, but we recommend writing these as zero. The case-insensitive base codes '=ACMGRSVTWYHKDBN' are mapped to [0,15] respectively with all other characters mapping to 'N' (value 15).

Omitted sequence, represented in SAM as '*', is represented by `l_seq` being 0 and `seq` and `qual` zero-length.

Base qualities are stored as bytes in the range [0,93], without any +33 conversion to printable ASCII. When base qualities are omitted but the sequence is not, `qual` is filled with 0xFF bytes (to length `l_seq`).

4.2.4 Auxiliary data encoding

Optional alignment fields are stored immediately after each other immediately following the `qual` field, and are included in `block_size`. Each field is represented as a two-character tag followed by a single type character and then its value, whose length is determined by the field's type.

Single character 'A' fields have a total length of 4 bytes, with the value represented as a single byte:

<i>tag</i>	A	char
------------	---	------

While all single (i.e., non-array) integer types are stored in SAM as 'i', in BAM any of 'cCsSiI' may be used together with the correspondingly-sized binary integer value, chosen according to the field value's magnitude.³³ Similarly floating point 'f' fields are represented as IEEE 754-2008 binary32 values. Thus BAM numeric fields have a total length of 4, 5, or 7 bytes:

<i>tag</i>	c	i8	(i.e., <code>int8_t</code>)	<i>tag</i>	i	<code>int32_t</code>
<i>tag</i>	C	u8	(i.e., <code>uint8_t</code>)	<i>tag</i>	I	<code>uint32_t</code>
<i>tag</i>	s	<code>int16_t</code>		<i>tag</i>	f	<code>float</code>
<i>tag</i>	S	<code>uint16_t</code>				

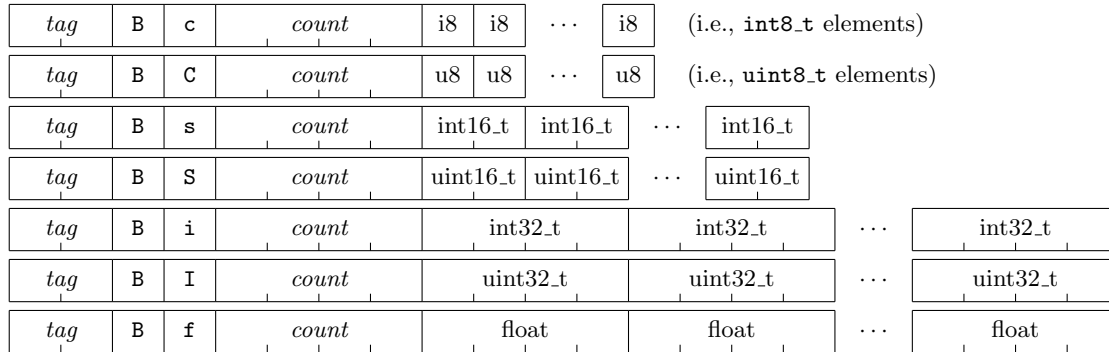
String fields and hex-formatted byte arrays are represented as NUL-terminated text strings:³⁴

<i>tag</i>	Z	char	char	...	char	NUL
<i>tag</i>	H	hex	hex	hex	hex	NUL

³³The signedness and size used for each integer value is an implementation choice, but is typically the smallest that suffices.

³⁴The BAM representation of 'H' field values as textual hexadecimal digits rather than binary data is for historical reasons. Modern applications may prefer to use 'B,C' array fields rather than 'H' fields.

The representation of a ‘B’ array field starts with a sub-type character similar to the numeric field types above and a *count* (`uint32_t`, but limited by memory and `block_size`) giving the number of elements in the array. The array elements follow, encoded as binary integers or IEEE floats sized according to the sub-type:



5 Indexing BAM

Indexing aims to achieve fast retrieval of alignments overlapping a specified region without going through the whole alignments. BAM must be sorted by the reference ID and then the leftmost coordinate before indexing.

This section describes the binning scheme underlying coordinate-sorted BAM indices and its implementation in the long-established BAI format. The CSI format documented elsewhere uses a similar binning scheme and can also be used to index BAM.³⁵

5.1 Algorithm

5.1.1 Basic binning index

The UCSC binning scheme was suggested by Richard Durbin and Lincoln Stein and is explained in Kent *et al.*³⁶ In this scheme, each bin represents a contiguous genomic region which is either fully contained in or non-overlapping with another bin; each alignment is associated with a bin which represents the smallest region containing the entire alignment. The binning scheme is essentially a representation of R-tree. A distinct bin uniquely corresponds to a distinct internal node in a R-tree. Bin A is a child of Bin B if the region represented by A is contained in B.

To find the alignments that overlap a specified region, we need to get the bins that overlap the region, and then test each alignment in the bins to check overlap. To quickly find alignments associated with a specified bin, we can keep in the index the start file offsets of chunks of alignments which all have the bin. As alignments are sorted by the leftmost coordinates, alignments having the same bin tend to be clustered together on the disk and therefore usually a bin is only associated with a few chunks. Traversing all the alignments having the same bin usually needs a few seek calls. Given the set of bins that overlap the specified region, we can visit alignments in the order of their leftmost coordinates and stop seeking the rest when an alignment falls outside the required region. This strategy saves half of the seek calls in average.

In the BAI format, each bin may span 2^{29} , 2^{26} , 2^{23} , 2^{20} , 2^{17} or 2^{14} bp. Bin 0 spans a 512Mbp region, bins 1–8 span 64Mbp, 9–72 8Mbp, 73–584 1Mbp, 585–4680 128kbp, and bins 4681–37448 span 16kbp regions. This implies that this index format does not support reference chromosome sequences longer than $2^{29} - 1$.

The CSI format generalises the sizes of the bins, and supports reference sequences of the same length as are supported by SAM and BAM.

5.1.2 Reducing small chunks

Around the boundary of two adjacent bins, we may see many small chunks with some having a shorter bin while the rest having a larger bin. To reduce the number of seek calls, we may join two chunks having the same bin if they are close to each other. After this process, a joined chunk will contain alignments with different bins. We need to keep in the index the file offset of the end of each chunk to identify its boundaries.

5.1.3 Combining with linear index

For an alignment starting beyond 64Mbp, we always need to seek to some chunks in bin 0, which can be avoided by using a linear index. In the linear index, for each tiling 16384bp window on the reference, we record the smallest file offset of the alignments that overlap with the window. Given a region [rbeg, rend), we only need to visit a chunk whose end file offset is larger than the file offset of the 16kbp window containing rbeg.

With both binning and linear indices, we can retrieve alignments in most of regions with just one seek call.

³⁵See CSIv1.pdf at <https://github.com/samtools/hts-specs>. This is a separate specification because CSI is also used to index other coordinate-sorted file formats in addition to BAM.

³⁶W. James Kent *et al.*, **The Human Genome Browser at UCSC**, *Genome Res.* 2002 12: 996–1006; doi:10.1101/gr.229102; PMID:12045153. See in particular *The Database*, p1003.

5.1.4 A conceptual example

Suppose we have a genome shorter than 144kbp. We can design a binning scheme which consists of three types of bins: bin 0 spans 0-144kbp, bin 1, 2 and 3 span 48kbp and bins from 4 to 12 span 16kbp each:

0 (0-144kbp)								
1 (0-48kbp)			2 (48-96kbp)			3 (96-144kbp)		
4 (0-16k)	5 (16-32k)	6 (32-48k)	7 (48-64k)	8 (64-80k)	9 (80-96k)	10	11	12

An alignment starting at 65kbp and ending at 67kbp would have a bin number 8, which is the smallest bin containing the alignment. Similarly, an alignment starting at 51kbp and ending at 70kbp would go to bin 2, while an alignment between [40k, 49k] to bin 0. Suppose we want to find all the alignments overlapping region [65k, 71k]. We first calculate that bin 0, 2 and 8 overlap with this region and then traverse the alignments in these bins to find the required alignments. With a binning index alone, we need to visit the alignment at [40k, 49k] as it belongs to bin 0. But with a linear index, we know that such an alignment stops before 64kbp and cannot overlap the specified region. A seek call can thus be saved.

5.2 The BAI index format for BAM files

Field	Description	Type	Value
magic	Magic string	char[4]	BAI\1
n_ref	# reference sequences	uint32_t	$< 2^{31}$
<i>List of indices (n=n_ref)</i>			
n_bin	# distinct bins (for the binning index)	uint32_t	≤ 37451
<i>List of distinct bins (n=n_bin)</i>			
bin	Distinct bin	uint32_t	≤ 37450
n_chunk	# chunks	uint32_t	limited ³⁷
<i>List of chunks (n=n_chunk)</i>			
chunk_beg	(Virtual) file offset of the start of the chunk	uint64_t	
chunk_end	(Virtual) file offset of the end of the chunk	uint64_t	
n_intv	# 16kbp intervals (for the linear index)	uint32_t	$\leq 2^{17}$
<i>List of intervals (n=n_intv)</i>			
ioffset	(Virtual) file offset of the first alignment in the interval	uint64_t	
n_no_coor (optional)	Number of unplaced unmapped reads (RNAME *)	uint64_t	

The index file may optionally contain additional metadata providing a summary of the number of mapped and placed unmapped read-segments per reference sequence, and of any unplaced unmapped read-segments.³⁸ This is stored in an optional extra metadata pseudo-bin for each reference sequence, and in the optional trailing n_no_coor field at the end of the file.

The pseudo-bins appear in the references' lists of distinct bins as bin number 37450 (which is beyond the normal range) and are laid out so as to be compatible with real bins and their chunks:

bin	Magic bin number	uint32_t	37450
n_chunk	# chunks	uint32_t	2
ref_beg	(Virtual) file offset of the start of reads placed on this reference	uint64_t	
ref_end	(Virtual) file offset of the end of reads placed on this reference	uint64_t	
n_mapped	Number of mapped read-segments for this reference	uint64_t	
n_unmapped	Number of unmapped read-segments for this reference	uint64_t	

The ref_beg/ref_end fields locate the first and last reads on this reference sequence, whether they are mapped or placed unmapped. Thus they are equal to the minimum chunk_beg and maximum chunk_end respectively.

³⁷The number of chunks in a single bin is effectively limited by available memory and in any case is typically a maximum of some thousands.

³⁸By *placed unmapped read* we mean a read that is unmapped according to its FLAG but whose RNAME and POS fields are filled in, thus "placing" it on a reference sequence (see Section 2). In contrast, *unplaced* unmapped reads have '*' and 0 for RNAME and POS.

5.3 C source code for computing bin number and overlapping bins

The following functions compute bin numbers and overlaps for a BAI-style binning scheme with 6 levels and a minimum bin size of 2^{14} bp. See the CSI specification for generalisations of these functions designed for binning schemes with arbitrary depth and sizes.

When these functions are called with regions representing unplaced unmapped reads, e.g., `reg2bin(-1, 0)`, they involve operations such as `(-1)>>14` which are undefined or implementation-defined in some programming languages. They must be implemented as if these operations use the common two's-complement semantics: `reg2bin(-1, 0) = 4680` and `reg2bins(-1, 0, ...)` returns `[0, 0, 8, 72, 584, 4680]`.

```
/* calculate bin given an alignment covering [beg,end) (zero-based, half-closed-half-open) */
int reg2bin(int beg, int end)
{
    --end;
    if (beg>>14 == end>>14) return ((1<<15)-1)/7 + (beg>>14);
    if (beg>>17 == end>>17) return ((1<<12)-1)/7 + (beg>>17);
    if (beg>>20 == end>>20) return ((1<<9)-1)/7 + (beg>>20);
    if (beg>>23 == end>>23) return ((1<<6)-1)/7 + (beg>>23);
    if (beg>>26 == end>>26) return ((1<<3)-1)/7 + (beg>>26);
    return 0;
}

/* calculate the list of bins that may overlap with region [beg,end) (zero-based) */
#define MAX_BIN (((1<<18)-1)/7)
int reg2bins(int beg, int end, uint16_t list[MAX_BIN])
{
    int i = 0, k;
    --end;
    list[i++] = 0;
    for (k = 1 + (beg>>26); k <= 1 + (end>>26); ++k) list[i++] = k;
    for (k = 9 + (beg>>23); k <= 9 + (end>>23); ++k) list[i++] = k;
    for (k = 73 + (beg>>20); k <= 73 + (end>>20); ++k) list[i++] = k;
    for (k = 585 + (beg>>17); k <= 585 + (end>>17); ++k) list[i++] = k;
    for (k = 4681 + (beg>>14); k <= 4681 + (end>>14); ++k) list[i++] = k;
    return i;
}
```

Appendix A Parsing region notation

Parsing region notation such as *name[:begin[-end]]* (in which omission of the outer bracketed portion indicates a request for the entire reference sequence) would be simple if *name* could not itself contain ‘:’ characters, but this is not the case. (No such notation containing an optional ‘:’ appears in the SAM format itself, but various tools use this notation as a convenient way for their users to specify regions of interest.)

The set of valid reference sequence names is usually already known when parsing this notation—for example, because the associated **@SQ** headers have already been encountered. Tools can use this set to determine unambiguously which colons could delimit a known-valid reference sequence name.

In pseudocode form, a string *str* can be parsed as follows:

```
consider the rightmost ‘:’ character, if any, of str
if str is of the form ‘prefix:NUM’ or ‘prefix:NUM-NUM’
    or generally ‘prefix:suffix’ for some plausible interval suffix
then
    if both prefix and str are in the known set then ...error: ambiguous representation
    else if prefix is in the known set then return (prefix, NUM...NUM)
    else if str is in the known set then return (str, entire sequence)
    else ...error: unknown reference sequence name

else
    ...either str does not contain a colon or the suffix is not plausibly numeric
    if str is in the known set then return (str, entire sequence)
    else ...error: unknown reference sequence name or invalid interval syntax
```

The check leading to “error: ambiguous representation” is important as it prevents confusing interpretations of actually ambiguous input. Typically the set of valid reference sequence names will not contain names that are prefixes of other names in the set, so in practice this error will not usually be encountered in non-malicious data.

Either in addition to this algorithm or as an alternative to it, tools can use additional delimiter characters to make an unambiguously parsable notation. We recommend a convention using curly brackets around the reference sequence name—*{name}[:begin[-end]]*—as being memorable, easily typed, unambiguous, and not expanded by most shells.

Appendix B SAM Version History

This lists the date of each tagged SAM version along with changes that have been made while that version was current. The key changes that caused the version number to change are shown in bold.

Additions and changes to the standard predefined tags are listed in the separate *Sequence Alignment/Map Optional Fields Specification*.³⁹

1.6: 28 November 2017 to current

- Add **SINGULAR** to the list of **@RG** PL header tag values. (May 2023)
- Clarify that **@RG** PI values are integers. (May 2023)
- Add **ELEMENT** and **ULTIMA** to the list of **@RG** PL header tag values. (Aug 2022)
- Clarify that header field tags must be distinct within each line, and that the ordering of both header fields and alignment optional fields is not significant. (Jun 2021)
- Clarify the meaning of TLEN when secondary alignments are present. (May 2021)
- Bin calculation changed for alignment records whose CIGAR strings consume no reference bases: like unmapped records, they are considered to have length one (rather than zero). (Jan 2021)

³⁹See Appendix A of *SAMtags.pdf* at <https://github.com/samtools/hts-specs>.

- Correct the description of index pseudo-bins, which previously stated that `ref_beg/ref_end`, then named `unmapped_beg/unmapped_end`, include only placed unmapped reads. (Jul 2020)
- Add `DNBSEQ` to the list of `@RG PL` header tag values. (Apr 2020)
- Add `@SQ TP` circular/linear topology header tag. (May 2019)
- **Restricted the allowable punctuation characters in reference sequence names** (in `@SQ SN`, `RNAME`, etc). The sets of characters allowed in `@SQ SN` and `@SQ AN` are now identical, which enlarges the previous `AN` set. (Jan 2019)

We recommend that implementations validating reference sequence names do so using the rules in Section 1.2.1; are more lenient for files declaring `@HD VN ≤ 1.5`; and validate `AN` only against these rules, not the previous more restrictive `AN` rules.

- Add `@HD SS` sorting details header tag. (Oct 2018)
- B array optional fields may have no entries—this was already representable in BAM, clarified that empty arrays are permitted in SAM too. (Jul 2018)
- Add `@SQ DS` header tag. (Jul 2018)
- Add `@RG BC` header tag. (Apr 2018)
- Permit UTF-8 in a few header tags. (Mar 2018)
- **Add support for CIGAR strings with more than 65,535 operations.** (Nov 2017)

1.5: 23 May 2013 to November 2017

- Add `@SQ AN` header tag, allowing only alphanumeric and ‘`*+.@_-`’ characters in its names. (Jul 2017)
- Add `@SQ AH` header tag. (Mar 2017)
- Auxiliary tags migrated to SAMtags document. (Sep 2016)
- Z and H auxiliary tags are permitted to be zero length. (Jun 2016)
- `QNAME` limited to 254 bytes (was 255). (Aug 2015)
- Generalise 0x200 flag bit as filtered-out bit. (Aug 2015)
- Add `@HD GO` for group order. (Mar 2015)
- Add `ONT` to the `@RG PL` and `@RG PM` header tags. (Mar 2015)
- Add meaning to reverse `FLAG` on unmapped reads. (Mar 2015)
- Document the `idxstats .bai` elements. (Nov 2014)
- Addition of `CSI` index. (Sep 2014)
- Add `@PG DS` header field. (Dec 2013)
- Document the BAM EOF byte values. (Dec 2013)
- Glossary of alignment types. (May 2013)
- Note that `PNEXT/RNEXT` points to next read, not segment. (May 2013)
- **Add SUPPLEMENTARY flag bit.** (May 2013)

1.4: 21 April 2011 to May 2013

- Add guide to using sequence annotations (CT/PT tags). (Mar 2012)
- Increase max reference length from 2^{29} to 2^{31} . (Sep 2011)
- Clarify @SQ M5 header tag generation. (Sep 2011)
- Describe padded alignments. (Sep 2011)
- Add @RG FO, KS header fields. (Apr 2011)
- Clarify chaining of PG records. (Apr 2011)
- Add B array auxiliary tag type. (Apr 2011)
- Permit IUPAC in SEQ and MD auxiliary tag. (Apr 2011)
- Permit QNAME “*”. (Apr 2011)

1.3: July 2010 to April 2011

- Add RG PG header field. (Nov 2010)
- Add BAM description and index sections. (Nov 2010)
- Add ‘=’ and ‘X’ CIGAR operations. (July 2010)
- Removal of FLAG letters. (July 2010)
- The SM header field, previously mandatory for @RG, is now optional. (July 2010)

1.0: 2009 to July 2010

Initial edition.